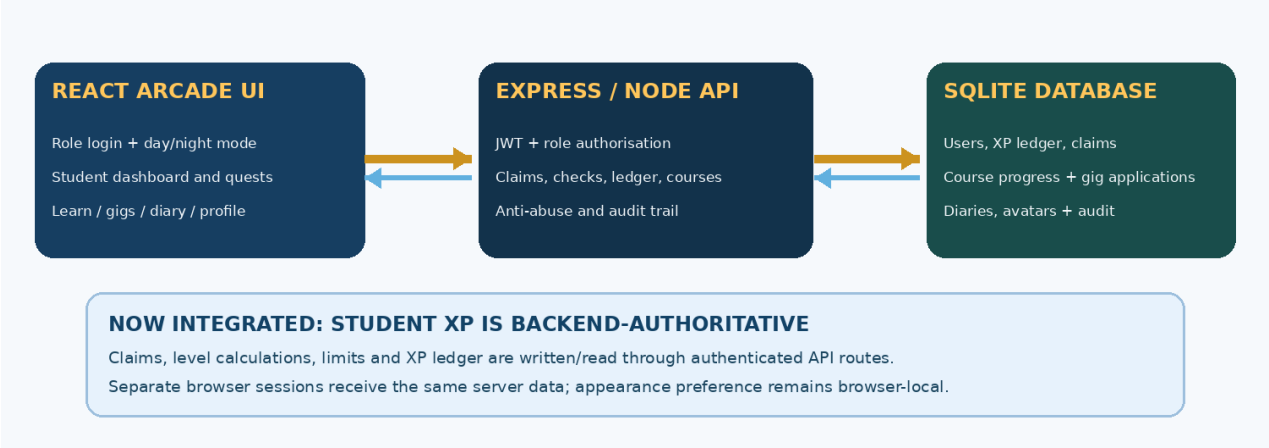
ALUMABLE • GAMIFICATION ENGINE

Final Technical Handover
Integrated Arcade Release

Student experience • XP → Level → Capability Unlock • Educator / Employer Portals



System architecture — updated integration design and data ownership

RELEASE IDENTIFICATION

Field	Handover status
Prepared	14 October 2026 • capstone team review draft
Frontend	Retro arcade, no opening poster; Light and Night modes
Backend	Express + SQLite, authenticated API; original XP rules preserved
Delivery state	Code changes supplied for validation on the team's Mac; no production deployment claimed
Required reviewer	Client/supervisor and team to verify and sign off

Status convention used throughout: "verified previously" refers to supplied screenshots from the prior release; "implemented—retest" identifies new code awaiting fresh installation and test evidence. No stakeholder approval has been assumed.

1. Purpose and original requirements

Alumable is a Melbourne education-technology platform connecting students and employers through short gigs and projects. The sponsor brief, attributed to David Yip (Founder & CEO), requests an MVP gamification engine that allows student activities to earn experience points, progress through levels, unlock platform capabilities and expose APIs to student/educator/employer portals.

BRIEF-ALIGNED OUTCOMES

Original objective	Delivered or current status
Award XP for activities and challenges	Backend rules and claim endpoint implemented; client-side XP replaced by API requests — retest
Educator / employer assignment and marking	Existing protected backend endpoints and portals retained
Level thresholds and feature unlocks	Backend authoritative rules; visualisation on student dashboard — retest
Consumable APIs	JWT-protected API exists; student data now read from server — retest
Future employer-sponsored bonus/reward paths	Architecture supports assignments/bonus XP; full sponsor rewards not evidenced
Integrate with MySQL / AWS / existing platform	Not yet completed or accepted; current database is SQLite sandbox

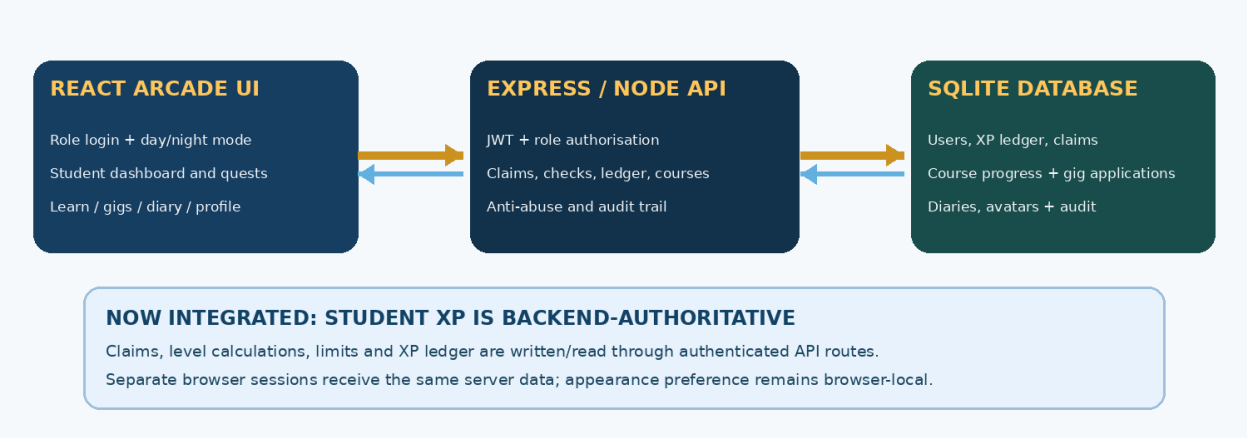
2. What is included in this handover

- Complete source ZIP containing the React arcade frontend, Express backend, SQLite schema, seed data and tests.
- ARCADE-README.md with local Mac installation, roles, security audit and cross-browser verification steps.
- INTEGRATION-CHECKLIST.md for recording release-specific validation results.
- This report with architecture, maintenance notes, limitation register and supporting interface visuals.

3. Technical components

Component	Technology / responsibility
Student web interface	React, Vite, responsive arcade theme and browser-local appearance preference
Login & authorisation	Email/password authentication; short-lived JWT; student/educator/employer route enforcement
Gamification API	Express + server-side rules for XP awards, levels, daily caps, rate limits and duplicate prevention
Data	SQLite ledger, claims, assignments, gigs, diary, avatar, course progress and audit log
Development runtime	Node.js 22.13+; localhost:5173 frontend and localhost:4000 backend

4. Architecture and integrated data flow



The backend is authoritative for student XP, activity eligibility and account-specific progress

The previous version independently stored student XP using localStorage key alumable-db-v1. A regular browser showed 1,690 XP while an Incognito session showed 1,250 XP, demonstrating a cross-session inconsistency. The updated StudentContext no longer imports or reads that local XP total. Instead, it loads authenticated API responses and uses server claim endpoints for XP changes.

KEY REQUEST PATHS

Method / route	Purpose
POST /api/auth/login	Authenticate a demo account and obtain a role-bound token
GET /api/me	Read authoritative total XP, level, unlocks, daily cap, avatar
GET /api/activities	Read activities with current server claim eligibility
POST /api/activities/:id/claim	Award XP or submit a claim for marking; validates daily/one-time/cap rules
GET /api/ledger; /api/audit	Read private XP history and a student's own audit entries
GET /api/courses	Read module counts from backend course-progress table
POST /api/activities/learn_module/claim	Claim XP and update course progress in the same DB transaction (with courseId)
GET /api/gigs; POST /api/gigs/:id/apply	Retrieve and manage gig applications; limit one active gig
PUT /api/me/avatar; /api/diary	Save avatar choices and manage private/shared reflections

The server's original XP calculation code is unchanged. New read endpoints and course-progress persistence were added to expose consistent values to the browser. Browser-local appearance and supplementary demonstration copy remain non-authoritative.

5. Installation and local operation

Use the root ARCADE-README.md as the executable setup guide. The following is a concise operational handover.

5.1 Backend

Open Terminal A in the extracted project folder and run:

- cd backend
- npm ci
- npm test
- npm start

Check http://localhost:4000/api/health and expect JSON {"ok":true}. First run seeds a local SQLite database in backend/data/alumable.db (unless DB_FILE is set). Do not include database files or .env secrets in the final source distribution.

5.2 Frontend

Open Terminal B in the extracted project root and run:

- npm install
- npm run build
- npm run dev

Open http://localhost:5173. The Vite development server is bound to localhost and requires port 5173, preventing the previous silent move to 5174/5175 that caused cross-origin confusion. A new frontend lockfile is generated during npm install because the dependency graph was updated for security.

5.3 Demo access and shutdown

Role	Local demo account
Student	priya@example.com / student-demo-1
Educator	educator@example.com / educator-demo-1
Employer	employer@example.com / employer-demo-1

Shut down each server using Control+C in its Terminal. Before restarting, check which processes hold 4000 or 5173 with lsof. Never terminate a PID without confirming the program and port first.

5.4 Configuration and security

- Use backend/.env.example and root .env.example for configuration; VITE_API_URL defaults to localhost:4000.
- Set a non-demo JWT_SECRET and restrict CORS origins before any internet-facing deployment.
- Do not publish accounts, demo passwords, local database, secrets, or private student information to production.
- The original sponsor target is a platform-integrated solution using MySQL/AWS; migration/deployment requires additional engineering and sponsor access.

6. User workflows and expected behaviour

Role	Workflow	Backend rule or limitation
Student	Log in → Home → Earn → claim available activity	Ledger records approved XP; server decides eligibility and amount
Student	Finish Learn module	Module count and approved XP are updated together under a DB transaction
Student	View levels / capabilities	Read level 1–5 thresholds and unlocks from server XP
Student	Apply for gig	Level requirements and one-active-gig rule enforced by API
Student	Write/refine diary; customise avatar	Diary private by default; avatar configuration stored per account
Educator	Create/assign activities; mark submitted work	JWT role check, marking rubric; rubric remains placeholder
Employer	Post gigs and assign work	Employer access cannot mark educator-owned assessment

7. Security and dependency treatment

The previous frontend npm installation reported one high-severity vulnerability, but the captured installation output did not name the affected package. The new release requests patched Vite and React Router dependency lines and removes the older compatibility package. Frontend package installation, a clean npm audit, and full browser regression testing still require verification on the team's Mac.

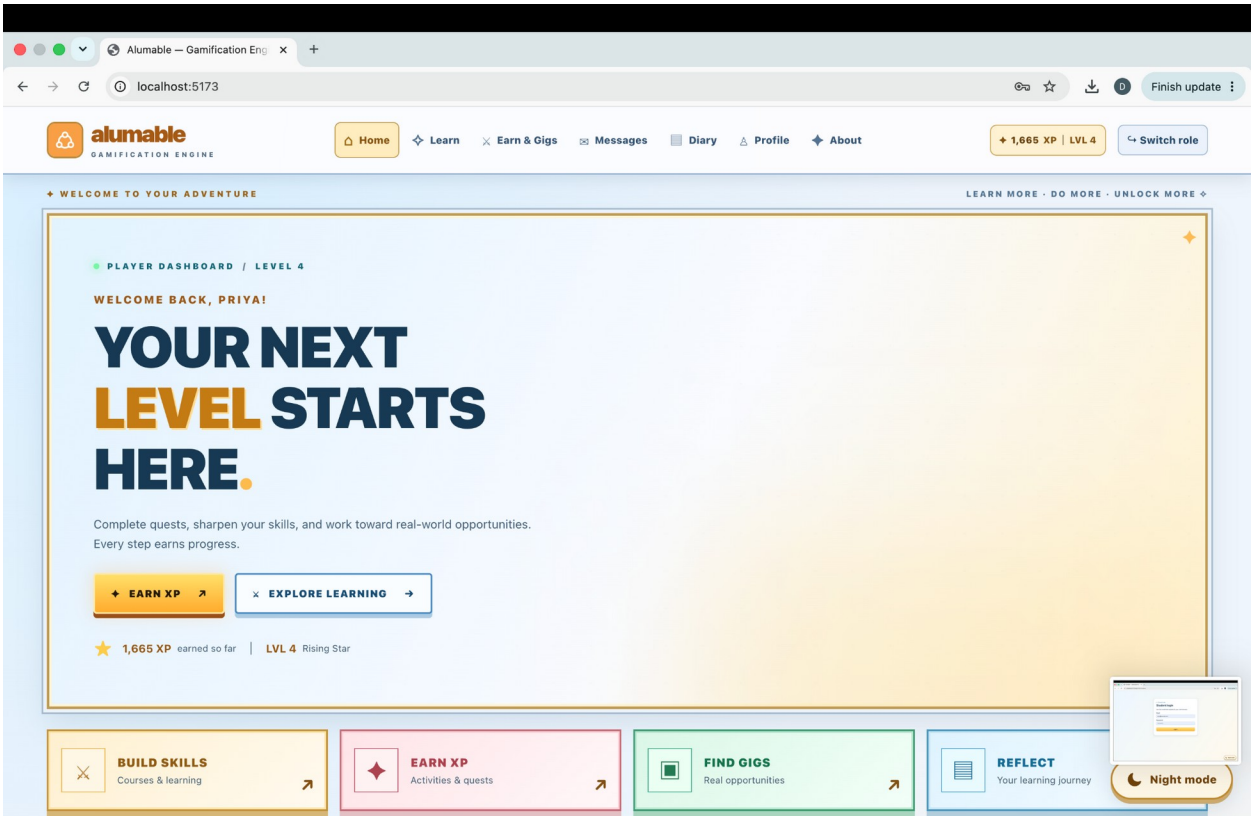
Security item	Evidence or handover action
Server role protection	Backend suite previously exercised JWT and role restrictions
Abuse prevention	Backend tested daily caps, duplicate claims, cooldowns, marking, rate limits
Dependency security	Run npm audit --audit-level=high after updated npm install; retain actual advisory results
Browser storage	XP removed from localStorage authority; night/light preference remains local intentionally
Transport / production hardening	HTTPS, database migration, secrets, backup, observability and production auth require further work

8. Testing evidence and release boundary

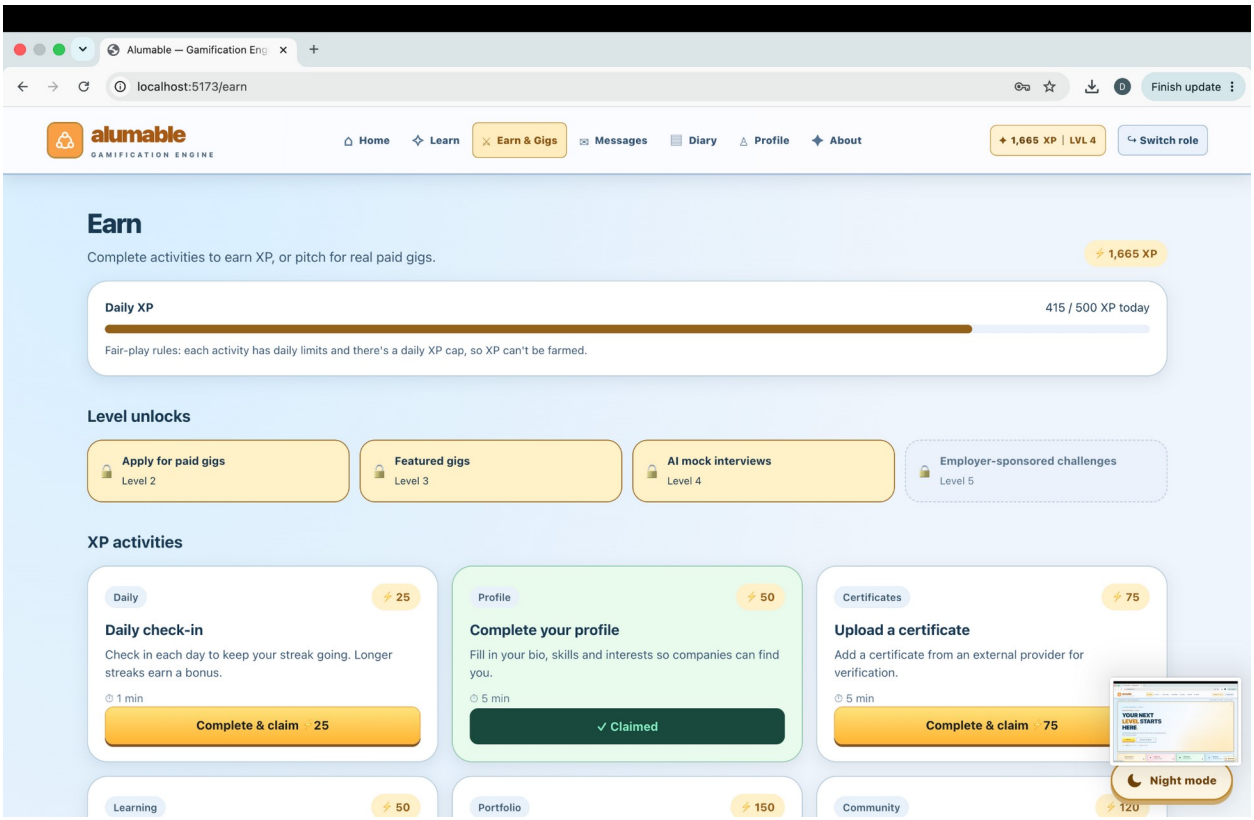
Test evidence	Observed result / scope
Backend npm test (prior release)	39 passed, 0 failed, 0 skipped — supplied terminal capture on 14 Oct 2026
Health check (prior release)	GET /api/health returned {"ok":true} — supplied browser capture
Student login / light mode (prior release)	User confirmed login, role screen, interface and theme persistence
Local browser XP claim (prior release)	1,665 → 1,690 XP, daily 415 → 440; persisted on refresh, NOT across Incognito
Updated backend tests	10 XP-rule unit tests passed locally; full API suite with new regression tests needs npm install and rerun
Updated frontend build + npm audit	Pending clean install and validation on the team's Mac
Updated cross-browser XP test	Pending; acceptance procedure included in INTEGRATION-CHECKLIST.md

9. Interface visuals and test evidence

The following interface visuals are from the user's browser session on 14 October 2026. They document the approved arcade design and the issue that motivated the API integration. They precede the code changes in this release, so they do not substitute for post-upgrade acceptance testing.

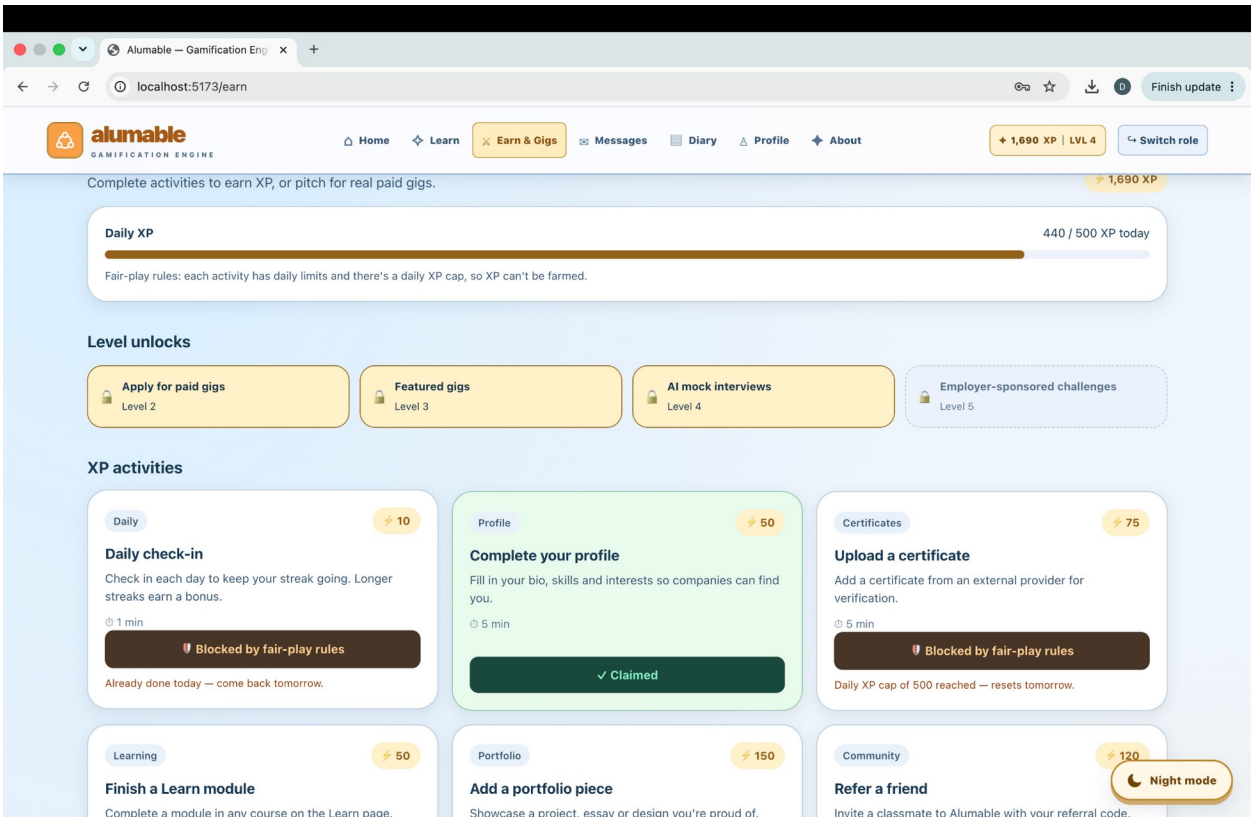


Interface visual 1 — Alumable student dashboard, Light mode; level 4, XP counter and navigation



Interface visual 2 — Earn page showing level unlocks, activity cards and daily XP bar

10. XP and backend evidence

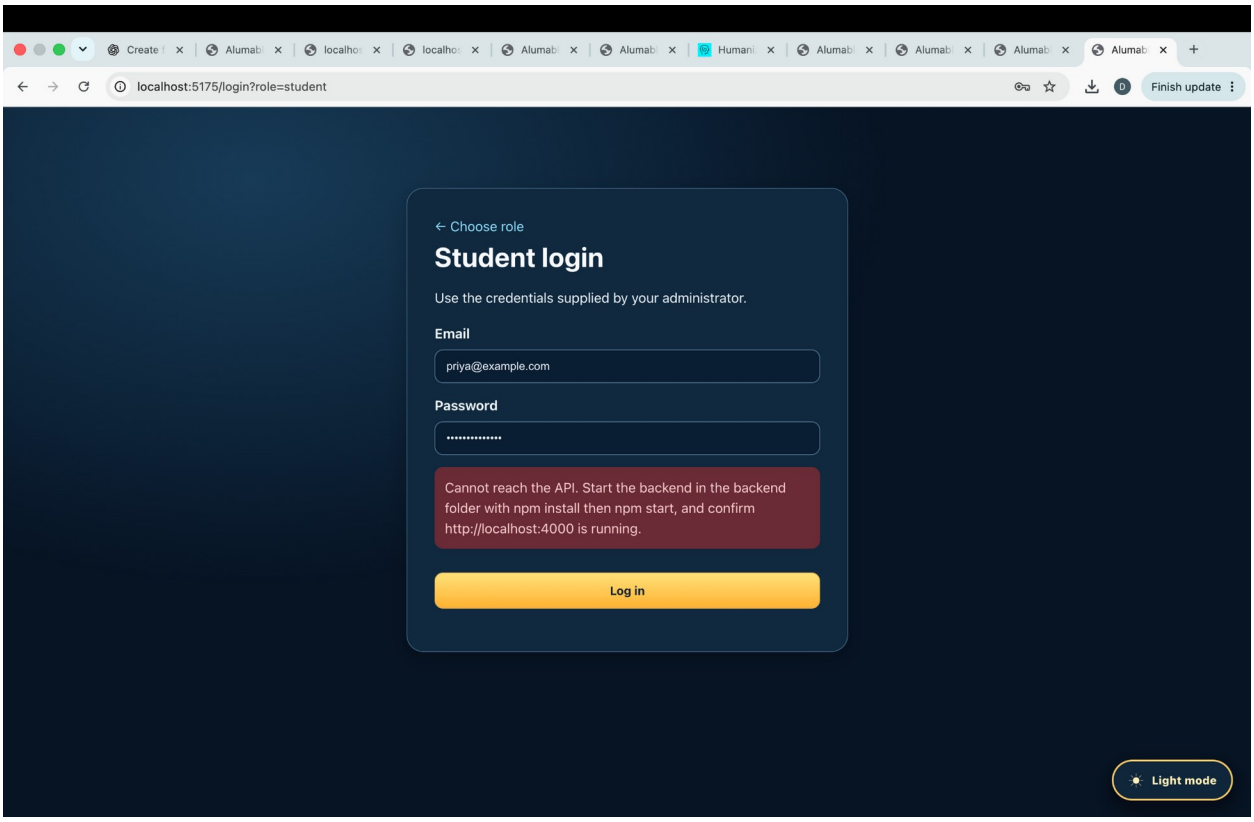


Interface visual 3 — 1,690 XP displayed after a local daily check-in; prior-release browser persistence

```
✔ capabilities unlock at their level (0.913375ms)
✔ streak bonus is 3 per day, capped at 5 days, daily activities only (0.077875ms)
✔ assignment bonus is added on top (0.038625ms)
✔ streak counts consecutive days and resets after a gap (0.657209ms)
✔ a day is the Melbourne day, not the UTC day (29.380959ms)
✔ claim rules: locked, one-time, daily, limit, cooldown, assignment, XP cap (0.243166ms)
ℹ tests 39
ℹ suites 0
ℹ pass 39
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 2802.001042
dersimaavci@Mac backend %
```

Testing evidence — terminal excerpt supplied by the team: 39 backend tests passed in the previous release

Additional interface visual — Night mode



Interface visual 4 — Night-mode student login before backend integration; theme retained in updated release

This image documents the agreed Night mode styling and role login. Updated-release login and XP synchronisation should be demonstrated again on the team's Mac using the checklist.

11. Limitations, remaining work and priorities

Priority	Outstanding issue	Acceptance requirement
P0	Re-run updated backend and frontend tests on Mac	Capture new test counts and build result, resolve failures
P0	Verify XP synchronisation across two browser sessions	Same server balance after claim, refresh, logout/login and restart
P0	Identify and clear the npm high-severity advisory	Capture npm audit output after updated dependency installation; resolve any remaining advisory
P1	Validate assigned work, marking and Learn module flow	Complete end-to-end tests including evidence, approval and XP awarding
P1	Confirm sponsor acceptance and rubric scoring	Document stakeholder feedback, criteria and sign-off
P2	Production integration	Plan MySQL/AWS, HTTPS, secrets, migration, backup and monitoring
P2	Improve demonstration-only content	Replace sample certificates/catalogue with verified data or explicit demo labels

12. Operational support / troubleshooting

Symptom	First check
Port 4000 already in use	lsof -nP -iTCP:4000 -sTCP:LISTEN; use existing server or stop recognised old one
Frontend opens on 5174/5175 or will not start	Release requires port 5173; close old Vite processes
Login cannot reach API	Verify /api/health, ports, root VITE_API_URL and backend CORS_ORIGIN
XP differs between browser sessions	Ensure running this updated release and same backend DB/account; ignore legacy localStorage XP
npm audit fails	Capture npm audit --json and inspect exact package/advisory before targeted update
Database write error	Confirm backend/data writable; backup SQLite before any schema/data migration

13. Handover checklist and acceptance

The following items require actual team/client confirmation before the handover can be labelled final or production-ready. This document deliberately leaves those approvals open.

☐ Package and source files transferred to the designated repository/team

☐ Fresh backend suite and frontend build results attached for this release

☐ Security audit completed and remaining findings documented

☐ Cross-browser XP persistence demonstration completed

☐ Role-based student, educator and employer workflows validated

☐ Original sponsor requirements reviewed against known SQLite/AWS integration gaps

☐ Placeholder marking criteria confirmed or clearly accepted as unfinished

☐ Handover meeting held and maintainer assigned

☐ Client/supervisor approval recorded

14. Release changes

Area	Previous release	Updated integrated release
Student XP	Browser localStorage (alumable-db-v1)	SQLite-ledger total via authenticated GET /api/me
Activities	Browser-calculated claim decisions	Server /api/activities checks and /claim writes
Progress history	Browser-saved ledger	Account-scoped backend /api/ledger
Course progress	Browser demo model	Server course-progress table and atomic award flow
Gigs	Local UI state	Server-backed apply/withdraw operations
Avatar	Local state	Account-backed avatar save with optional wardrobe validation
Security deps	React Router 7 compatibility + Vite 8.3.0	React Router 8.4 + Vite >=8.3.4, audit pending
Appearance	Night/light toggle stored locally	Preserved; intended per-device preference

15. Source record

Prepared using the following supplied evidence: Project Brief 2026 – Alumable Gamification Engine.pdf (pp. 1–3); Assessment 3 Submission rubric; Assessment 4 Rubric; Student Group Project Charter template (unsigned); Alumable_Arcade_Light_Dark_No_Poster.zip; backend README, DECISIONS.md, source/tests; 14 Oct 2026 team screenshots (backend test, health response, login, dashboard, XP claim).

The group charter supplied is a blank template; no signatures, individual assignments, client feedback or sponsor approval are inferred from it.

Handover review sign-off — Technical reviewer: _____ Date: _____ Client/sponsor: _____ Date: _____